Problem Statement: Insurance Policy Management System

(Using Inheritance, Runtime Polymorphism, Array of Base Class Pointers, Utility Class with Static Functions, and RTTI)

Design an Insurance Policy Management System using Object-Oriented Programming concepts in C++.

System Description

An insurance company provides different types of insurance policies. All policies share some common attributes, but each policy type provides additional features that are unique to that policy.

The system must store objects of different policy types in a single array of base class pointers and use runtime polymorphism to call common overridden functions. Additionally, RTTI must be used to safely downcast the base pointer and invoke special methods that are available only in derived classes.

Base Class: Policy

Create a base class named Policy.

Data Members:

- int policyNumber
- string policyHolderName
- double sumInsured
- double basePremium

Member Functions:

All the following functions must be declared as virtual:

- virtual void accept()

Accept common policy details.

- virtual void display()

  Display common policy details.

- virtual double calculatePremium()

  Calculate and return the premium amount.

- virtual destructor

  Required to ensure proper cleanup of derived class objects.

The base class must not contain any derived-specific special feature methods.

Derived Classes

Create the following classes derived from Policy. Each derived class must override all virtual functions from the base class.

Class HealthPolicy (Derived from Policy)

Additional Data Members:

- int age
- bool criticalIllnessCover

Overridden Functions:

- accept()

  Must first call base class accept() and then accept additional data members.
- display()

  Must first call base class display() and then display additional data members.

- calculatePremium()

  Must override and implement its own premium calculation logic.

Premium Calculation Rule:
basePremium + (age * 100)
If criticalIllnessCover is true, add 5000 extra to premium.

Derived-Specific Method (Not in Base Class):

- void provideCashlessFacility()

  Displays hospital network and cashless treatment details.

Class VehiclePolicy (Derived from Policy)

Additional Data Members:

- string vehicleType
- int vehicleAge

Overridden Functions:

- accept()
- display()
- calculatePremium()

Premium Calculation Rule:
basePremium + (vehicleAge * 500)
If vehicleAge is greater than 10 years, add additional risk charge of 2000.

Derived-Specific Method:

- void provideRoadsideAssistance()

  Displays emergency support and towing services details.

Class LifePolicy (Derived from Policy)

Additional Data Members:

- int policyTerm
- double bonusRate

Overridden Functions:

- accept()

- display()
- calculatePremium()

Premium Calculation Rule:
basePremium + (policyTerm * 200)

Derived-Specific Method:

- void displayNomineeBenefits()

  Displays nominee and maturity benefit details.

Array of Base Class Pointers

In main():

- Declare an array:

  Policy* policyArr[5];

- Ask the user to choose policy type for each index:

  1 for HealthPolicy
  2 for VehiclePolicy
  3 for LifePolicy

- Dynamically allocate the selected object using new.
- Store the derived class object address in the base class pointer array.
- After program execution, properly delete all dynamically allocated objects.

Utility Class (Static Only)

Create a class named PolicyUtility.

Rules:

- The class must contain only static member functions.

- No object of this class should be created.

Required Static Functions

1. acceptPolicy(Policy* pptr)
   Calls the overridden accept() method using runtime polymorphism.

2. displayPolicy(Policy* pptr)
   Calls the overridden display() method.

3. displayPremium(Policy* pptr)
   Calls the overridden calculatePremium() method and displays the result.

4. invokeSpecialFeature(Policy* pptr)
   This function must use RTTI (dynamic_cast) to:

   - Check the actual object type at runtime.
   - Downcast the base pointer safely.
   - Invoke the appropriate derived-specific method:

If object is HealthPolicy → call provideCashlessFacility();
If object is VehiclePolicy → call provideRoadsideAssistance() ;
If object is LifePolicy → call displayNomineeBenefits();

RTTI Requirement

- Use of dynamic_cast for safe downcasting.
- Why at least one virtual function is required in the base class.
- Difference between:
  - Runtime polymorphism (virtual function call through base pointer) ○

  RTTI-based invocation of derived-only methods

- Why derived-specific methods cannot be accessed directly using base class pointer.

SOLUTION:

File Location and Code

Header File

./Policy.h

```
#pragma once
#include<iostream>
using namespace std;
class Policy {
private:
	int policyNumber;
	string policyHolderName;
	double sumInsured;
	double basePremium;
public:
	Policy();
	Policy(int policyNumber, string policyHolderName, double sumInsured, double
basePremium);
	virtual void accept();
	virtual void display();
	virtual double calculatePremium() = 0;

	double getbasePremium();
};
```

./LifePolicy.h

```
#pragma once
#include"Policy.h"

class LifePolicy : public Policy {
private:
```

```
	int policyTerm;
	double bonusRate;
public:
	void accept();
	void display();
	double calculatePremium();
	void displayNomineeBenefits();
};
```

./HealthPolicy.h

```
#pragma once
#include"Policy.h"

class HealthPolicy :public Policy {
private:
	int age;
	bool criticalIllnessCover;
public:
	//HealthPolicy();
	void accept();
	void display();
	double calculatePremium();
	void provideCashlessFacility();
};
```

./VehiclePolicy.h

```
#pragma once
#include"Policy.h"

class VehiclePolicy:public Policy {
private:
	string vehicleType;
	int vehicleAge;
public:
	void accept();
	void display();
	double calculatePremium();
```

```
	void provideRoadsideAssistance();
};
```

Resource File

./policy_src.cpp

```
#include"Policy.h"
#include "HealthPolicy.h"


Policy::Policy() {
	policyNumber = 0;
	policyHolderName = "Na";
	sumInsured = 0.0;
	basePremium = 0.0;
}
Policy::Policy(int policyNumber, string policyHolderName, double sumInsured, double
basePremium) {
	this->policyNumber = policyNumber;
	this->policyHolderName = policyHolderName;
	this->sumInsured = sumInsured;
	this->basePremium = basePremium;
}
void Policy::accept() {
	cout << "Enter policy number\n";
	cin >> policyNumber;
	cout << "Enter POlicy Holder Name : \n";
	cin >> policyHolderName;
	cout << " Sum Insured : \n";
	cin >> sumInsured;
	cout << "Base Premium : \n";
	cin >> basePremium;
}
void Policy::display() {
	cout << "\nPolicy Number " << policyNumber <<
		"\nPolicy Holder Name : " << policyHolderName;
	cout << "\nSum Insured: " << sumInsured << "\nBase Premium  : " << basePremium <<
endl;
```

```
}



double Policy::getbasePremium() {

	return basePremium;
}
```

./LifePolicy_src.cpp

```
#include"LifePolicy.h"
#include"Policy.h"

void LifePolicy::accept() {
	Policy::accept();
	cout << "Enter policy Term: \n";
	cin >> policyTerm;
	cout << "Enter Bonus Rate: \n";
	cin >> bonusRate;
}
void LifePolicy::display() {
	Policy::display();
	cout << " PolicyTerm : " << policyTerm<< endl;
	cout << "Bonus Rate : " << bonusRate << endl;
}
double LifePolicy::calculatePremium() {
	double premium = getbasePremium() + (policyTerm * 200);

	return premium;

}
void LifePolicy::displayNomineeBenefits() {
	cout << "\n--- Nominee and Maturity Benefits ---\n";
	cout << "Nominee Details: Ensure your nominee information is up to date for smooth
claim settlement.\n";
	cout << "Maturity Benefits: The lump sum amount payable upon the policy reaching its
term, including accrued bonuses.\n";
}
```

./HealthPolicy_src.cpp

```
#include"HealthPolicy.h"

double HealthPolicy::calculatePremium() {
	double premium = getbasePremium() + (age * 100);
	if (criticalIllnessCover)
	{
		premium += 5000;
	}
	return premium;
}
void HealthPolicy::accept() {
	Policy::accept();
	cout << "Enter age: \n";
	cin >> age;
	cout << "Enter Critical illnesscover : \n";
	cin >> criticalIllnessCover;

}

void HealthPolicy::display() {
	Policy::display();
	cout << "\n Age is: " << age <<
		"\n Critical illnesscover : " << criticalIllnessCover;

}

void HealthPolicy::provideCashlessFacility() {
	cout << "\n--- Cashless Facility Details ---\n";
	cout << "Hospital Network: Our policy includes a wide network of partner hospitals.\n";
	cout << "For cashless treatment, please present your policy number and ID card at a
network hospital.\n";
	cout << "Consultation and claim processing will be handled directly with the insurer.\n";
}
```

./VehiclePolicy_src.cpp

```
#include"Policy.h"
#include"VehiclePolicy.h"

void VehiclePolicy::accept(){
	Policy::accept();
	cout << "Enter vehicleType :\n";
	cin >> vehicleType;
	cout << "Enter  vehicleAge : \n";
	cin >> vehicleAge;
}
void VehiclePolicy:: display() {
	Policy::display();
	cout << "Vechile Type : " << vehicleType << endl;
	cout << "Vehicle age :" << vehicleAge << endl;
}
double VehiclePolicy::calculatePremium(){
	double premium = getbasePremium() + (vehicleAge * 500);
	if (vehicleAge > 10) {
		cout << "Aditionall risk charge = 2000\n";
		premium += 2000;
	}
	return premium;
}
void VehiclePolicy::provideRoadsideAssistance(){
	cout << "\n--- Roadside Assistance Details ---\n";
	cout << "Emergency Support: 24/7 hotline for immediate assistance.\n";
	cout << "Towing Services: Free towing to the nearest network garage within a 50km
radius.\n";
	cout << "Minor on-site repairs are also covered.\n";
}
```

Source/Client File

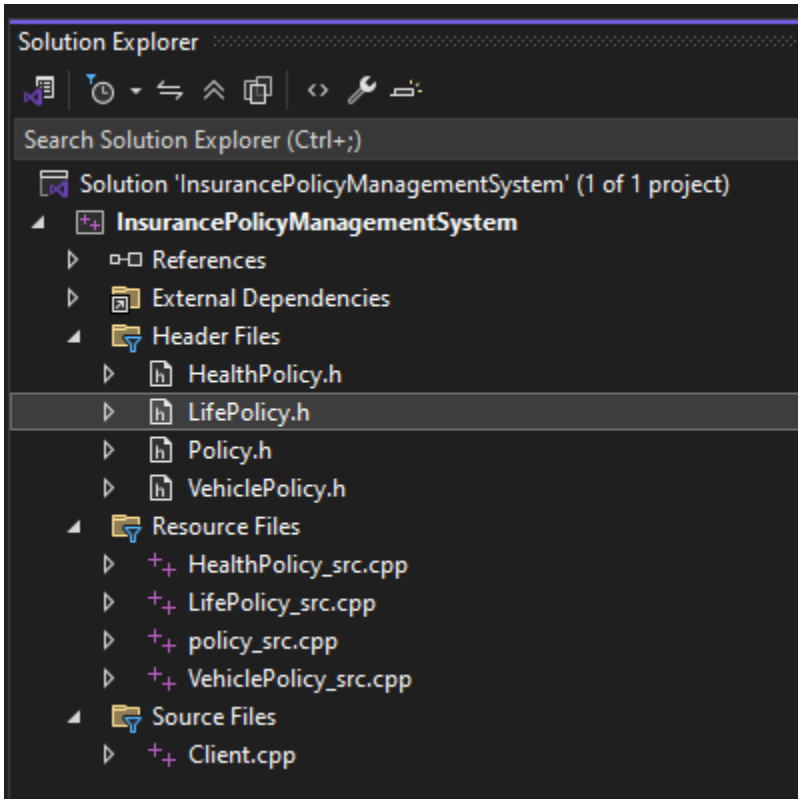
./client.cpp

```
#include"Policy.h"
#include"HealthPolicy.h"
#include"LifePolicy.h"
#include"VehiclePolicy.h"

class PrinttoScreen {
public:
	static void acceptPolicy(Policy* pptr) {
		pptr->accept();
	}
	static void displayPolicy(Policy* pptr) {
		pptr->display();
	}

	static void displayPremium(Policy* pptr) {
		pptr->calculatePremium();
	}
	static void invokeSpecialFeature(Policy* pptr)
	{
		if (typeid(pptr) == typeid(HealthPolicy)) {

			HealthPolicy* hptr = dynamic_cast<HealthPolicy*>(pptr);
			hptr->provideCashlessFacility();

		}
		if (typeid(pptr) == typeid(VehiclePolicy)) {

			VehiclePolicy* vptr = dynamic_cast<VehiclePolicy*>(pptr);
			vptr->provideRoadsideAssistance();

		}
		if (typeid(pptr) == typeid(LifePolicy)) {

			LifePolicy* lptr = dynamic_cast<LifePolicy*>(pptr);
			lptr->displayNomineeBenefits();

		}
```

```
        }
};


int main() {

    Policy* policyArr[5];


    int choice;
    for (int i = 0; i < 5; ++i) {
        cout << "\nEnter choice for Policy " << i + 1 << ":\n";
        cout << "1 for HealthPolicy\n";
        cout << "2 for VehiclePolicy\n";
        cout << "3 for LifePolicy\n";
        cout << "Choice: ";

        // Basic input validation loop
        while (!(cin >> choice) || choice < 1 || choice > 3) {
            cout << "Invalid choice. Please enter 1, 2, or 3: ";
            cin.clear(); // Clear error flags
            cin.ignore(10000, '\n'); // Discard bad input
        }

        switch (choice) {
        case 1:
            policyArr[i] = new HealthPolicy();
            PrinttoScreen::acceptPolicy(policyArr[i]);
            break;
        case 2:
            policyArr[i] = new VehiclePolicy();
            PrinttoScreen::acceptPolicy(policyArr[i]);
            break;
        case 3:
            policyArr[i] = new LifePolicy();
            PrinttoScreen::acceptPolicy(policyArr[i]);
            break;
        default:
            // This case should not be reached due to validation loop
            policyArr[i] = nullptr;
            break;
        }

        // Check if allocation was successful
```

```
        if (policyArr[i] != nullptr) {
            cout << "Policy " << i + 1 << " created successfully.\n";
        }
    }
    for (int i = 0; i < 5; i++) {
        PrinttoScreen::invokeSpecialFeature(policyArr[i]);
    }
    for (int i = 0; i < 5; i++) {
        PrinttoScreen::displayPolicy(policyArr[i]);
    }

    for (int i = 0; i < 5; i++) {
        delete[] policyArr[i];
    }
}
```

STRUCTURE OF PROGRAM / FILE



OUTPUT

Microsoft Visual Studio Debug Console

```
Enter choice for Policy 1:
1 for HealthPolicy
2 for VehiclePolicy
3 for LifePolicy
Choice: 1
Enter policy number
01
Enter POlicy Holder Name :
Vikas
 Sum Insured :
1000000
Base Premium :
3000
Enter age:
35
Enter Critical illnesscover :
true
Policy 1 created successfully.

Enter choice for Policy 2:
1 for HealthPolicy
2 for VehiclePolicy
3 for LifePolicy
Choice: Invalid choice. Please enter 1, 2, or 3: 2
Enter policy number
02
Enter POlicy Holder Name :
Vivek
 Sum Insured :
20000000
Base Premium :
4000
Enter vehicleType :
Sedan
Enter  vehicleAge :
12
Policy 2 created successfully.

Enter choice for Policy 3:
1 for HealthPolicy
2 for VehiclePolicy
3 for LifePolicy
Choice: 3
Enter policy number
03
Enter POlicy Holder Name :
Nutan
 Sum Insured :
30000000
Base Premium :
```

```
03
Enter POlicy Holder Name :
Nutan
 Sum Insured :
30000000
Base Premium :
5000
Enter policy Term:
2
Enter Bonus Rate:
3
Policy 3 created successfully.

Enter choice for Policy 4:
1 for HealthPolicy
2 for VehiclePolicy
3 for LifePolicy
Choice: 1
Enter policy number
04
Enter POlicy Holder Name :
Muskan
 Sum Insured :
30000000
Base Premium :
3000
Enter age:
25
Enter Critical illnesscover :
false
Policy 4 created successfully.

Enter choice for Policy 5:
1 for HealthPolicy
2 for VehiclePolicy
3 for LifePolicy
Choice: Invalid choice. Please enter 1, 2, or 3: 1
Enter policy number
05
Enter POlicy Holder Name :
Bumrah
 Sum Insured :
4000000
Base Premium :
5000
Enter age:
30
Enter Critical illnesscover :
true
Policy 5 created successfully.
```

```
Policy Number 1
Policy Holder Name : Vikas
Sum Insured: 1e+06
Base Premium  : 3000

 Age is: 35
 Critical illnesscover : 0
Policy Number 2
Policy Holder Name : Vivek
Sum Insured: 2e+07
Base Premium  : 4000
Vechile Type : Sedan
Vehicle age :12

Policy Number 3
Policy Holder Name : Nutan
Sum Insured: 3e+07
Base Premium  : 5000
 PolicyTerm : 2
Bonus Rate : 3

Policy Number 4
Policy Holder Name : Muskan
Sum Insured: 3e+07
Base Premium  : 3000

 Age is: 25
 Critical illnesscover : 0
Policy Number 5
Policy Holder Name : Bumrah
Sum Insured: 4e+06
Base Premium  : 5000

 Age is: 30
 Critical illnesscover : 0
D:\Vikas\Day6\InsurancePolicyManagementSystem\x64\Debug\InsurancePolicyManageme
ntSystem.exe (process 14128) exited with code -1073741819 (0xc0000005).
Press any key to close this window . . .
```